Who says what

21CSE306P Applied Generative AI · GENEVOE'26 · Batch 1, Team 11

7 slides
4 speakers

THE RUNNING ORDER

Anukool Kashyap — slides 1 and 2
Opens, names the team, states the problem and what we built to solve it.

Ananth Vignesh — slide 3
Walks the pipeline. This is the centre of the presentation; everything else refers back to it.

Shaik Mahammad Sahid Babu — slides 4 and 5
Answers the expert's comments, then names the models, the techniques and what comes next.

Kodali Shashank — slides 6 and 7
Explains how it is actually implemented, shows the live result and closes.

THE ONE SENTENCE EVERYONE MUST BE ABLE TO SAY

> "We turn a YouTube video — or a whole course playlist — into a summary, a timeline and a mind map, where **every single line can be traced back to the moment in the video where it was actually said.**"

THE NUMBERS FROM OUR REAL RUN (UNIT 5 — GRAPH THEORY)

5 videos in one playlist · **6,223 words** of transcript · **38 chunks** indexed together · **29 chunks** retrieved · faithfulness **0.688** · relevance **0.756** · compression **5.9×** · **7** sentences flagged as weakly grounded · **0** prompt-injection, safety or bias hits.

HAND-OVER LINES

> **Anukool → Ananth:** "Ananth will show you how that actually works, stage by stage."
>
> **Ananth → Sahid:** "Sahid will take the feedback we got and what we changed because of it."
>
> **Sahid → Shashank:** "Shashank will show it running."

ANUKOOL KASHYAP

RA2411003010166 · Computing Technologies

slides
1 & 2
about 75 seconds

WHAT YOU SAY

"Good morning. We are Team 11, Batch 1. Our project is **AI-Powered Video Summarization for Smart Learning.**"

"The problem is simple. A one-hour recorded lecture cannot be searched. If you want one idea from it, you watch it again. And a course is not one video — it is a playlist, so the same topic is taught across several lectures."

"Normal AI summarisers do help, but they state things the lecture never said. A student cannot tell which part is real, and cannot find where it was said. **That is the problem we fix.**"

"Our solution has four parts: we only let the model see text that was actually retrieved from the transcript; we handle a whole playlist as one course; we check every timestamp against the transcript; and if the video does not cover something, the system says so instead of guessing."

CONCEPTS YOU OWN

Grounding
The model is only allowed to use text we hand it from the transcript. It is not answering from what it learned during training.

Hallucination
When a language model states something confident and false. This is the thing our whole design is built to prevent.

Grounded refusal
If the answer is not in the retrieved text, the system replies "not covered in this video". Saying nothing is better than saying something wrong.

IF ASKED

"Why not just use ChatGPT?"
A general chatbot answers from memory and cannot point to the minute in the lecture. Ours answers only from this transcript and cites the timestamp, so a student can verify it.

"Who is this for?"
Students revising recorded lectures and online courses — exactly the situation where you have five hours of video and one exam tomorrow.

WHAT YOU SAY — FOLLOW THE ARROWS, LEFT TO RIGHT, THEN BACK

"This is the pipeline. Six stages run in a fixed order, and three layers sit across all of them."

"**One, ingestion.** We pull the timed transcript from YouTube. For a playlist we list every video first, then pull each transcript."

"**Two, chunking.** We cut the transcript into pieces of about a thousand characters with two hundred characters of overlap. The overlap matters — without it, an idea explained across a cut gets split in half. We cut on transcript boundaries, so every chunk keeps a real start and end time."

"**Three, embedding.** Each chunk becomes a 384-dimension vector using MiniLM, and goes into ChromaDB. Two chunks about the same idea end up close together, even in different words."

"**Four, retrieval.** We fetch twelve candidates, re-rank them, and keep the best five. That is the only text the model will ever see."

"**Five, generation.** open-mistral-nemo writes the summary from those five chunks, and returns it as structured JSON so we can parse it reliably."

"**Six, output.** Key points, clickable timings and a mind map."

"Layers seven to nine — error handling, red teaming and evaluation — apply to every stage. **They run in a fixed order and nothing here changes a setting on its own.**"

CONCEPTS YOU OWN

Embedding
Turning text into a list of numbers so that similar meanings sit close together. Ours has 384 numbers per chunk.

Vector store / ChromaDB
A database that searches by meaning instead of by keyword, using cosine similarity between vectors.

Re-ranking
Scoring the candidates again before choosing: 75% vector similarity, 20% word overlap, 5% position in the video. Vector search alone misses exact terms; word overlap alone misses paraphrases.

Encoder vs decoder
MiniLM is an encoder — it reads text and produces a vector. Mistral is a decoder — it writes new text. We use both, for different jobs.

SHAIK MAHAMMAD SAHID BABU

RA2411028010108 · Networking & Communications

slides
4 & 5
about 75 seconds

WHAT YOU SAY — SLIDE 4, THE FEEDBACK

"We were told three things, and we acted on all three."

"The grounding constraint and the faithfulness check were called positive, so we kept them — and we now report faithfulness, relevance and compression **per video and as a course average.**"

"We were asked to handle transcript quality. Auto-captions carry filler and noise, so we strip that before chunking, we validate every timestamp against the retrieved text, and any sentence we cannot fully ground we **flag rather than hide** — seven of them on our last run."

"And we were asked to support multiple videos, because a course is a playlist. **That is built and working** — Shashank will show it running in a moment."

WHAT YOU SAY — SLIDE 5, THE MODEL

"The generative model is **open-mistral-nemo**. Embeddings are all-MiniLM-L6-v2, and the vector store is ChromaDB. The technique is Retrieval-Augmented Generation with prompt engineering — **we did not fine-tune anything.**"

CONCEPTS YOU OWN

Retrieval-Augmented Generation (RAG)
Retrieve the relevant text first, then let the model write using only that text. The knowledge comes from the video, not from the model's memory.

Faithfulness
How much of the summary is actually supported by the retrieved transcript. Ours is 0.688. Be honest: it is a word-overlap measure, not a true entailment check.

Compression
How much shorter the summary is than the transcript. 5.9× on our run.

IF ASKED

"Why not fine-tune the model on lectures?"
Fine-tuning bakes facts into weights, where you cannot audit them or point to a source. With RAG, changing the video changes the answer immediately, with no retraining and no training data.

"Is 0.688 good?"
It is a proxy, so we read it as a trend, not a grade. What matters more is that the seven weak sentences are shown to the user instead of being quietly kept.

KODALI SHASHANK

RA2411028010093 · Networking & Communications

slides
6 & 7
about 90 seconds

WHAT YOU SAY — SLIDE 6, IMPLEMENTATION

"Three layers. The RAG layer: MiniLM into ChromaDB into Mistral, with a fixed six-stage order and the top five chunks chosen by re-ranking."

"The prompt engineering layer: a role prompt, a grounding constraint, a JSON schema and one worked example."

"The validation layer: timestamps that do not appear in the retrieved text are thrown away, and the scores are **reviewed offline by a person** — they never change a setting automatically."

"The limits are on the slide and they are real: we depend on the transcript, auto-captions carry errors, and faithfulness is a proxy measure."

WHAT YOU SAY — SLIDE 7, THE DEMO

"This is a real run on a five-video playlist, Unit 5, Graph Theory. 6,223 words, 38 chunks indexed together."

"Here it answers a question using **three different lectures at once** — you can see the citations: V4 at zero minutes, V1 at 8:25, V2 at 3:55. Each one is clickable and jumps to that moment."

"And the evaluation tab: faithfulness 0.688, relevance 0.756, compression 5.9×, zero injection, safety and bias hits, seven weakly grounded sentences flagged. **Thank you.**"

CONCEPTS YOU OWN

Course mode
All videos of a playlist go into one index. A question searches every lecture, with a cap of two chunks per video so one long lecture cannot crowd out the rest.

Timestamp validation
The model may invent a time. We check each one against the chunks we actually retrieved, allowing 15 seconds, and discard the rest.

Red teaming
Four automatic screens on every run: prompt injection, content safety, bias, and grounding.

Everyone must know these

Questions can go to any of the four of you

shared
Q & A

The one trap — do not call this agentic

Our system is a **pipeline**: six stages that run in the same fixed order every time. It does not plan, it does not choose tools, it does not act on its own and it does not improve itself. The evaluation scores are read by a person afterwards and never change a setting automatically.

Never say: agent, agentic, autonomous, orchestrator, planner, tool calling, self-improving, decides for itself. **Say instead:** pipeline, fixed sequence of stages, cross-cutting layers, reviewed offline by a human.

WHY THIS IS GENERATIVE AI, NOT JUST ML

ML → DL

Classical ML needs hand-built features. Deep learning learns the features itself — that is what makes MiniLM able to place meaning in vector space.

DL → NLP

Applied to language: tokenising, removing stop words for search, measuring similarity between pieces of text. Stages 2 and 4 are NLP.

NLP → Transformer

Attention lets a model weigh every word against every other word. MiniLM is the encoder half; it reads and represents.

Transformer → GenAI

The decoder half writes new text. Mistral produces a summary that appears nowhere in the transcript — that is generation, not extraction.

LIKELY QUESTIONS

"Why does it say top-5 but 29 chunks were retrieved?"
Top-5 is for a question. For a summary we sweep the whole video with four broad probes and force the first and last chunk in, so nothing at the start or end is missed — across five videos that adds up to 29.

"What is the prompt engineering, exactly?"
Four techniques, all in **prompts.py**: a role prompt, a grounding constraint that forbids outside knowledge, a JSON schema for structured output, and one worked example. It is already in the code, not typed live in the chat.

"What happens with a video that has no captions?"
It is skipped and reported, and the rest of the playlist continues. One bad video does not sink a thirty-video course.

"What is the biggest weakness?"
We are only as good as the transcript. Auto-captions mis-hear technical terms, and our faithfulness score is a word-overlap proxy rather than a real entailment check. Both are on the slide.

"Can it handle other languages?"
Not yet — it is the first item in our future scope.